

5分でわかるPHPの型システム

Introducing PHP's type system in 5 minutes

にゃんだーすわん @tadsan / USAMI Kenta うさみけんた

#tfcon

お前誰よ

ヾ(〃><)ノ

- ・ ピクシブ株式会社 pixiv事業本部 (2012～)
 - ・ pixivやピクシブ百科事典などを担当
- ・ TechFeed公認エキスパートNo.27
- ・ PHPカンファレンス / PHPerKaigi スタッフ
- ・ Emacs PHP Modeメンテナ
- ・ phpusers-ja/Emacs JPなどのコミュニティ
- ・ spcamp 2010 プログラミング言語組



#tfcon

PHPは簡単な言語だから
上級テクニックなんてないよ笑

PHPは簡単な言語だから
上級テクニックなんてないよ笑
言語が簡単なので素朴に書くとエントロピーが無限に増大する

PHPの型にどのような
印象を持っていますか？

脆弱性 ゆるふわ 弱い型 動的
クソザコ 型なし 意味不明 弱い
自動変換 貧弱 Perlっぽい 適当

PHPについてのパブリックイメージ

~~脆弱性~~ ~~ゆるふわ~~ ~~弱い型~~ 動的?
クソザコ ~~型なし~~ 意味不明 ~~弱い~~
自動変換 貧弱 Perlっぽい ~~適当~~

PHPについての認識は概ね間違い

PHPの値は型タグと共用体の 組に過ぎませんか？


```

288 typedef union _zend_value {
289     zend_long      lval;           /* long value */
290     double         dval;           /* double value */
291     zend_refcounted *counted;
292     zend_string     *str;
293     zend_array      *arr;
294     zend_object     *obj;
295     zend_resource    *res;
296     zend_reference   *ref;
297     zend_ast_ref     *ast;
298     zval             *zv;
299     void             *ptr;
300     zend_class_entry *ce;
301     zend_function     *func;
302     struct {
303         uint32_t w1;
304         uint32_t w2;
305     } ww;
306 } zend_value;

```

```

+-----+
| Zend Engine                                     |
+-----+
| Copyright (c) Zend Technologies Ltd. (http://www.zend.com) |
+-----+
| This source file is subject to version 2.00 of the Zend license, |
| that is bundled with this package in the file LICENSE, and is    |
| available through the world-wide-web at the following url:      |
| http://www.zend.com/license/2_00.txt.                            |
| If you did not receive a copy of the Zend license and are unable to |
| obtain it through the world-wide-web, please send a note to      |
| license@zend.com so we can mail you a copy immediately.         |
+-----+
https://github.com/php/php-src/blob/php-8.1.0/Zend/zend\_types.h

```

```

308 struct _zval_struct {
309     zend_value      value;          /* value */
310     union {
311         uint32_t type_info;
312         struct {
313             ZEND_ENDIAN_LOHI_3(
314                 zend_uchar type,      /* active type */
315                 zend_uchar type_flags,
316                 union {
317                     uint16_t extra;    /* not further specified */
318                 } u)
319             } v;
320         } u1;
321         union {
322             uint32_t next;             /* hash collision chain */
323             uint32_t cache_slot;       /* cache slot (for RECV_INIT) */
324             uint32_t opline_num;       /* opline number (for FAST_CALL) */
325             uint32_t lineno;           /* line number (for ast nodes) */
326             uint32_t num_args;         /* arguments number for EX(This) */
327             uint32_t fe_pos;           /* foreach position */
328             uint32_t fe_iter_idx;      /* foreach iterator index */
329             uint32_t property_guard;    /* single property guard */
330             uint32_t constant_flags;   /* constant flags */
331             uint32_t extra;            /* not further specified */
332         } u2;
333     };

```

PHPの値は型タグと共用体の 組に過ぎませんか？

過言です

極論はさておいてPHP ランタイムの型はこれだけ

言語処理系の内部実装について 「Rubyソースコード完全解説」 が参考になる

bool int float string
array object null
resource (callable iterable mixed)

JSONで扱える型 + 特殊な型がある

無限の可能性を持つ **array**

Pythonでのlist/dict
RubyのArray/Hashを併せ持つ

ユニオン型 **A | B**

AまたはB
(複雑な式は書けない)

交叉型 A&B

AおよびB
(複雑な式は書けない)

null許容型 ?A

Aまたはnull
デフォルトで許容しない

謎のPHP用語 **mixed**

いわゆるany
(PHPマニュアルの記述に由来)

型宣言できない特殊型 **resource**

ファイルポインタなど
(徐々にobjectに置き換え)

型はランタイムに
確実に保証される

```
1 <?php
2
3 function add(int $a, int $b): int
4 {
5     return $a + $b;
6 }
7
8 echo add('0', '1');
```

整数ではない文字列は
型宣言通りに変換

Output for 7.4.0 - 7.4.29, 8.0.1 - 8.0.19, 8.1.0 - 8.1.6

1

```
1 <?php
2
3 function add(int $a, int $b): int
4 {
5     return $a + $b;
6 }
7
8 $result = add('a', '1');
```

整数ではない文字列は
TypeErrorになる

Output for 8.0.1 - 8.0.18, 8.1.0 - 8.1.6

Fatal error: Uncaught TypeError: add(): Argument #1 (\$a) must be of type int, string given, c

Stack trace:

#0 /in/JAKKB(8): add('a', '1')

#1 {main}

thrown in /in/JAKKB on line 3

Process exited with code 255.


```
1 <?php
2
3 declare(strict_types=1);
4
5 function add(int $a, int $b): int
6 {
7     return $a + $b;
8 }
9
10 echo add('0', '1');
```

ファイル単位で
strict_types=1指定で
自動変換を抑制

Output for 8.0.1 - 8.0.19, 8.1.0 - 8.1.6

Fatal error: Uncaught TypeError: add(): Argument #1 (\$a) must be of type int, string given,

Stack trace:

#0 /in/hYTqv(10): add('0', '1')

#1 {main}

thrown in /in/hYTqv on line 5

Process exited with code 255.

どちらにせよ受け取った
関数の側ではintであることが
保証される

```
1 <?php
2
3 function add(int $a, int $b): int
4 {
5     var_dump([get_debug_type($b) => $b]);
6     return $a + $b;
7 }
8
9 echo add('0', '1');
```

PHPでは実行時に
型情報がとれる

Output for 8.0.1 - 8.0.19, 8.1.0 - 8.1.6

```
array(1) {
    ["int"]=>
    int(1)
}
1
```



```
1 <?php
2
3 function add(int|float $a, int|float $b): float
4 {
5     var_dump([
6         get_debug_type($a) => $a,
7         get_debug_type($b) => $b,
8     ]);
9     return $a + $b;
10 }
11
12 echo add('0.1', '1');
```

ユニオン型宣言で
適切に変換される

Output for 8.0.1 - 8.0.19, 8.1.0 - 8.1.6

```
array(2) {
    ["float"]=>
    float(0.1)
    ["int"]=>
    int(1)
}
1.1
```


classとinterfaceを ユーザー定義型として利用可能

```
1 <?php
2
3 function printYmd(DateTime $dt): void
4 {
5     echo get_debug_type($dt), $dt->format('Y-m-d');
6 }
7
8 printYmd(new DateTime);
```

型宣言にクラス名を指定できる

Output for 8.0.1 - 8.0.19, 8.1.0 - 8.1.6

DateTime2022-05-13

```
1 <?php
2
3 function printYmd(DateTimeInterface $dt): void
4 {
5     echo get_debug_type($dt), $dt->format('Y-m-d');
6 }
7
8 printYmd(new DateTime);
9 printYmd(new DateTimeImmutable);
```

インターフェイスも指定できる

Output for 8.0.1 - 8.0.19, 8.1.0 - 8.1.6

DateTime2022-05-13

DateTimeImmutable2022-05-13


```
1 <?php
2
3 function printYmd(DateTimeInterface $dt): void
4 {
5     echo get_debug_type($dt), $dt->format('Y-m-d');
6 }
7
8 printYmd(new DateTime);
9 printYmd(new DateTimeImmutable);
```

void型宣言で戻り値を
返さないことを明示できる

= その関数には副作用がある

```
1 <?php
2
3 function redirect_to(string $url, int $status = 300): never
4 {
5     http_response_code($status);
6     header("Location: {$url}");
7     exit;
8 }
9
10 if ($cond == 'bad') {
11     redirect_to(ERROR);
12 }
13
14 // 正常な場合の処理
15
```

never型宣言でメソッドが
正常終了しないことを宣言
= 異常状態が後続に漏れない

WSGI/Rack/PSGI相当のWeb アプリケーションのI/Fも 型宣言ベースで定義されている

1	<?php	MIT License
2		Copyright (c) 2018 PHP-FIG
3	namespace Psr\Http\Message;	Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
4		
5	interface ResponseFactoryInterface	
6	{	The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
7	/**	THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
8	* Create a new response.	
9	*	
10	* @param int \$code HTTP status code; defaults to 200	
11	* @param string \$reasonPhrase Reason phrase to associate with status code	
12	* in generated response; if none is provided implementations MAY use	
13	* the defaults as suggested in the HTTP specification.	
14	*	
15	* @return ResponseInterface	
16	*/	
17	public function createResponse(int \$code = 200, string \$reasonPhrase = ''): ResponseInterface;	
18	}	

MIT License - Copyright (c) 2018 PHP-FIG

<https://github.com/php-fig/http-factory/blob/1.0.1/src/ResponseFactoryInterface.php>

型はランタイムに 確実に保証される

それはわかった

型が実行時に保証される点で
PHPは他の動的言語とは
一線を画しているといえる

要するにPHPは
statically typed
静的型付き…ってコト！？

型なしはどこから来るの？

動的言語を型なしと呼ぶのも
別に誹謗中傷というわけではない

```
1 <?php
2
3 # /url?id=12345 を期待する
4 $id = $_GET['id']; //=> int?
5
```

PHPではURLクエリを
グローバル変数から取り出せる

```
1 <?php
2
3 # 現実
4 # /url?id=abc          === 'abc'
5 # /url?id[]=123        === '123'
6 # /url?id[key]=123     === ['key' => 123]
7 # /url?id[k1][k2][k3]=123 = ['k1' => ['k2' => ['k3' => 123]]]
8 $id = $_GET['id'];
9 //=> string|array
10
```


いろんなところから湧き出す 型の特定できないデータ

```
1 <?php
2
3 // JSONデータのデコード結果
4 $data = json_decode($str);
5
6 // eval (PHPコードの動的実行)
7 $data = eval($code);
8
9 // DBのクエリ結果
10 $data = $db->query($sql);
11
```


arrayは無限の可能性を秘める (悪い意味で)

```
1 <?php
2
3 function searchUser(int $user_id): array
4 {
5     // ...
6 }
7
8 $user = searchUser(11);
9
10  $user[???] ← どんなキーで格納されてるのか不明
11
```

扱う対象が複数になったのに 型に反映されない

```
1 <?php
2
3 function searchUsers(array $user_ids): array
4 {
5     // ...
6 }
7
8 $users = searchUsers([11, 22, 33]);
9
10 $users[???] ← どんなキーで格納されてるのか不明
11
```

PHPの組み込みの型機能では
どんなに型推論しても
静的に型が定まらない

array_map

(PHP 4 >= 4.0.6, PHP 5, PHP 7, PHP 8)

array_map — 指定した配列の要素にコールバック関数を適用する

説明

arrayの中に何が
入ってるのかわからん

```
array_map(?callable $callback, array $array, array ...$arrays): array
```

几帳面に型宣言だけを追うと
人間にはわかる型も機械は理解できない

```
1 <?php
2
3 function first(array $array, mixed $default): mixed
4 {
5     return isset($array[0]) ? $array[0] : $default;
6 }
7
8 $a = ['string']; // => array
9 $v2 = first($a, ''); // => mixed
10
```

無限の可能性を秘めたarrayを
放置するとコード全体の
エントロピーが増大し
プロジェクトは熱的死を迎える

ランタイムに保証 されない型を書こう

Meet The Next Member of Your Team!

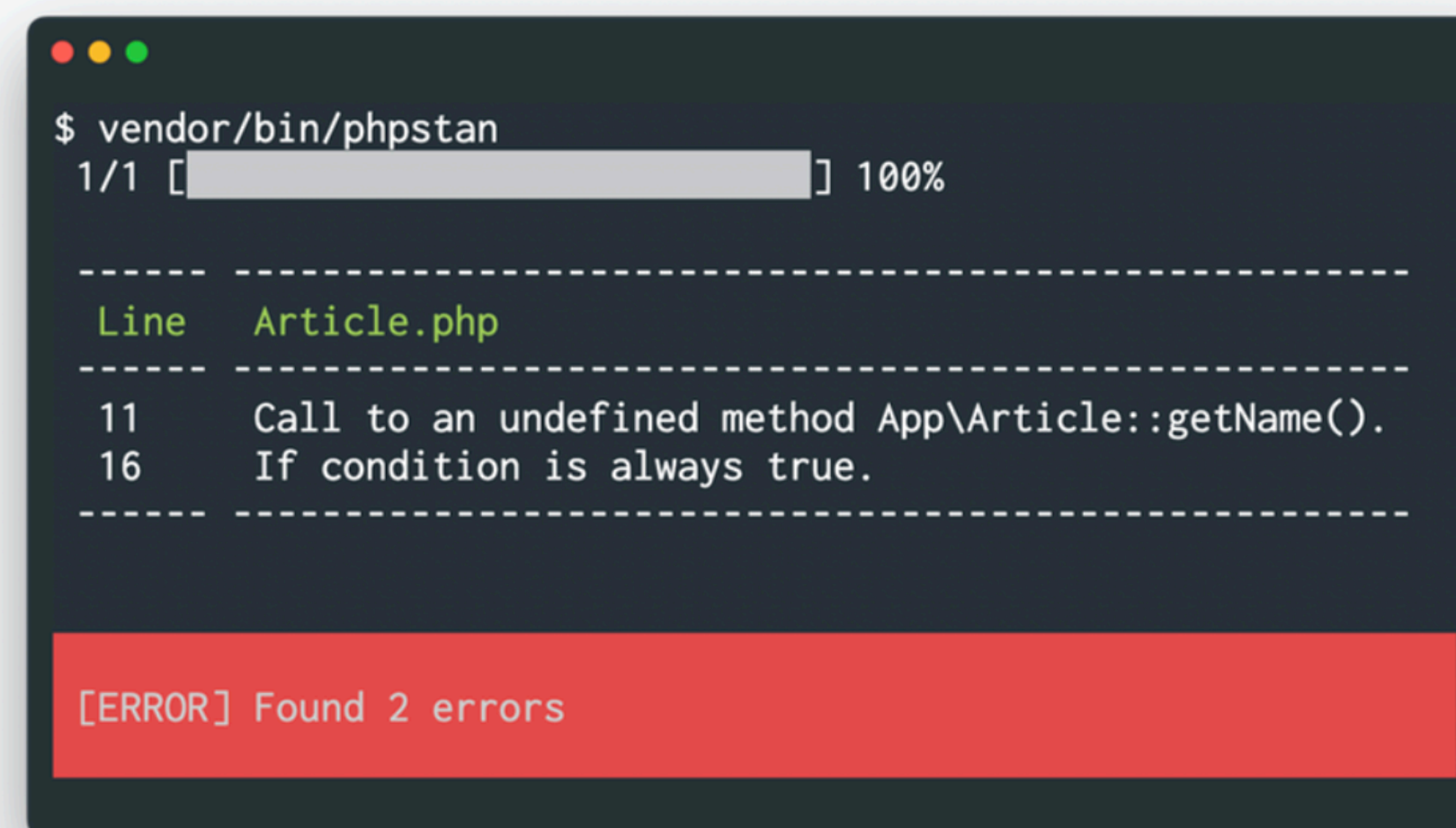
PHPStan finds bugs in your code without writing tests. It's open-source and free.

[Get Started](#)[Try It Online](#)

Find bugs before they reach production

PHPStan scans your whole codebase and looks for both obvious & tricky bugs. Even in those rarely executed if statements that certainly aren't covered by tests.

You can run it on your machine and in CI to prevent those bugs ever reaching your customers in production.



```
$ vendor/bin/phpstan
1/1 [████████████████████] 100%

-----
Line   Article.php
-----
11     Call to an undefined method App\Article::getName().
16     If condition is always true.
-----

[ERROR] Found 2 errors
```

Give PHP the love it deserves

It's easy to make great things in PHP, but bugs can creep in just as easily. Psalm is a free & open-source static analysis tool that helps you identify problems in your code, so you can sleep a little better.

Psalm helps people maintain a wide variety of codebases – large and small, ancient and modern. On its strictest setting it can help you prevent almost all type-related runtime errors, and enables you to take advantage of safe coding patterns popular in other languages.

Psalm also fixes bugs automatically, allowing you to improve your code without breaking a sweat.

```
1 <?php
2
3 /**
4  * @return array<string>
5  */
6 function takesAnInt(int $i) {
7     return [$i, "hello"];
8 }
9
10 $data = ["some text", 5];
11 takesAnInt($data[0]);
12
13 $condition = rand(0, 5);
14 if ($condition) {
15 } elseif ($condition) {}
```

[↙ Expand](#)[🔗 Get link](#)



PhpStorm

超高速でスマートなPHP IDE

今すぐダウンロード

無料30日間体験版

PhpStormを選択する理由

PHP開発に プロの生産性と楽しさを

▶ 概要を見る

array-shapes

/** */ のブロックに型を書く
array{key: type} で構造体
のような型を記述できる

```
1 <?php
2
3 /**
4  * @return array{name: string, age: int}
5  */
6 function searchUser(int $user_id): array
7 {
8     return ['name' => 'miku', 'age' => 16];
9 }
10
11 $user = searchUsers(11);
12 $user['name']; // => string
13 $user['age']; // => int
```

array<T>

array<key, value>

list<value> のように型を書ける

```
3  /**
4   * @return array<int, array{name: string, age: int}>
5   */
6  function searchUsers(array $user_ids): array
7  {
8      return [
9          11 => ['name' => 'miku', 'age' => 16],
10         22 => ['name' => 'rin', 'age' => 14],
11         33 => ['name' => 'len', 'age' => 14],
12     ];
13 }
14
15 foreach (searchUsers([11, 22, 33]) as $user) {
16     $user['name']; // => string
17     $user['age']; // => int
18 }
```

@template

型パラメータで戻り値を
可変にできる

```
1 <?php
2
3 /**
4  * @template T
5  * @param list<T> $array
6  * @param T $default
7  * @return T
8  */
9 function first(array $array, mixed $default): mixed
10 {
11     return isset($array[0]) ? $array[0] : $default;
12 }
13
14 $a = ['string']; // => array
15 $v2 = first($a, ''); // => mixed
```


@template

ツール互換性のために
@phpstan- を付けてもよい

```
1  <?php
2
3  /**
4   * @template T
5   * @phpstan-param list<T> $array
6   * @phpstan-param T $default
7   * @phpstan-return T
8   */
9  function first(array $array, mixed $default): mixed
10 {
11     return isset($array[0]) ? $array[0] : $default;
12 }
13
14 $a = ['string']; // => array
15 $v2 = first($a, ''); // => mixed
```


アプリケーションの エントロピーを減少させて 快適に開発しよう